GPU Buffer Pool for Product-Key Memory Layers

Unmesh Chakravarty (ukc), Sumukh Venkatesh (sumukhv)

Summary

We are going to build a GPU buffer pool that lets product-key memory layers use value tables that are larger than GPU memory by keeping the full table in host memory and caching hot rows in device memory. Our CUDA implementation on GHC RTX 2080s will combine a concurrent GPU cache with batch-level request deduplication, skew-aware replacement, and choosing for every batch whether to read directly from host memory (zero-copy reads) or to gather all missed reads and allocate a contiguous buffer for all them (bulk transfer). We'll evaluate it against the case where all the values fits in GPU, the case where all the values are in CPU and the GPU fetches with cudaMemcpy, and CUDA Unified Memory.

Background

Product-key memory layers add a very large table of learned vectors that each token reads sparsely.

Each token's query vector q is split into two halves, q_1 and q_2. Each half is scored against its own table of n subkeys, C_1 and C_2 (each key is a pair (i, j) with subkey i from C_1 joined with subkey j from C_2, giving us $|K| = n^2$ keys). We have

$$s_1 = C_1 q_1, \quad s_2 = C_2 q_2, \quad I_1 = \text{top-k}(s_1), \quad I_2 = \text{top-k}(s_2)$$

We want the best k pairs. We know the best k pairs must have each half be in their respective top-k list. Thus, the k^2 combinations of the two top-k lists are scored by summing, and the best k become the selected set S. The output is a softmax-weighted sum of rows of the value table V

$$S = \text{top-k}\{s_1[i] + s_2[j] \,:\, i \in I_1, j \in I_2\}, \quad y = \sum_{(i,j)\in S} w_{ij} V[i \cdot n + j]$$

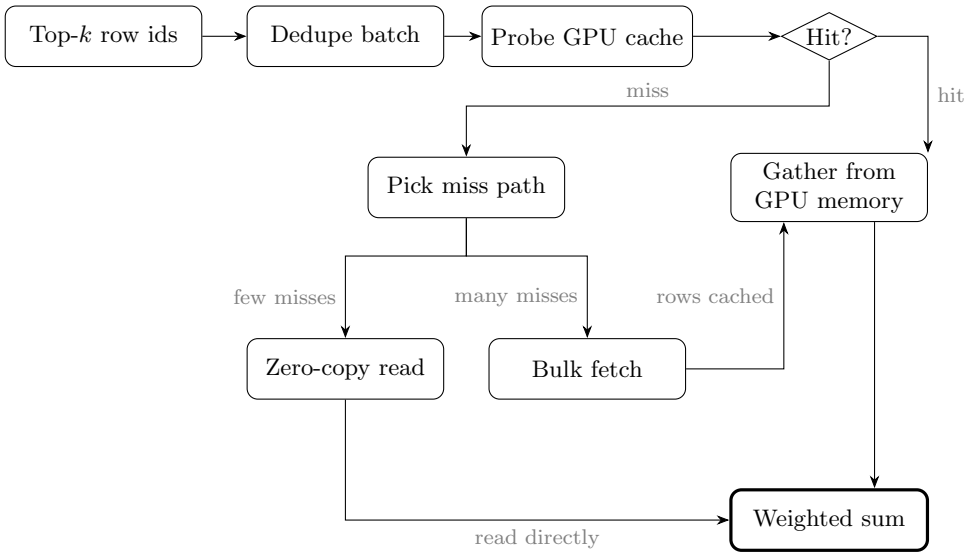
With h heads and a batch of T tokens, oen layer gathers $T \cdot h \cdot k$ rows. This computation is memory bounded since we are only doing a few multiply-adds

per byte read.

These layers add capacity by adding rows, so we want a larger table. A table of 2^{24} rows of 1 KB is 16 GB, which is twice the 8 GB on GHC RTX 2080 (Meta's memory layers scale to 128 billion memory parameters which is about 256 GB). In the published product-key systems we found, the table stays in device memory (Meta's memory layers shard it across GPUs when it doesn't fit in one).

DeepSeek's Engram offloads its lookup tables to host memory with negligible overhead because its addresses are hashes of the input tokens and can be computed and prefetched. Product-key addresses on the other hand depend on the hidden state at that layer and are known only after the top-k selection runs. This means every miss is on the critical path.

We will build a buffer pool in GPU memory that caches hot rows of a value table held in host memory. After top-k selection produces a batch's row ids, the batch is deduplicated, each unique row is looked up in a concurrent GPU hash table, hits are gathered from device memory, and misses are handled by either direct zero-copy reads of host memory or by bulk transfers into the cache. The weighted sum then reads every row from device memory or the zero-copy.



Every stage is data parallel over thousands of lookups per batch. This includes sorting and deduplicating ids, probing the hash table, gathering rows, and issuing transfers. Within a gather, one warp handles one row. Each of its 32 threads reads 32 contiguous bytes. We will fix value width to be 512, so 512 values $\div$ 32 threads = 16 values per thread, 16 values $\times$ 2 bytes = 32 bytes, and

32 threads $\times$ 32 bytes = 1 KB which is the whole row. Together, the threads cover the row's 1 KB with adjacent aligned reads which the GPU merges into a few wide memory transactions. Transfers can also overlap with gathers where the GPU transfers missed rows on one CUDA stream while gathering rows that hit on another CUDA stream.

Challenge

Moving data between host and device memory is a lot slower than reading data in device memory. Addresses are only known at the last minute. The access pattern is heavily skewed and there is contended access.

Row ids are known only after the layer's top-k selection, so misses are on the critical path (unless if we can predict them?).

Eviction shouldn't remove a row the current batch still needs. We will pin rows in use with per-slot pin counts (same idea as page guards in database buffer pool).

Inserting a fetched row, updating the hash table, and evicting a victim must stay consistent while other threads are also probing the table.

Access is irregular and skewed since popular rows are read way more often than others and can be reused both within a batch and across batches.

The ratio of communication to computation is high. The weighted sum does one multiply-add per 2 bytes read. This means throughput is bounded by memory.

It is hard to make use of locality. Unique rows are scattered across a table of millions of rows, so there is no spatial locality. The only thing we can do is reuse popular rows based on the skewed access pattern.

Within a warp, some lookups hit and some miss. Hits read device memory while misses are much slower. A bad kernel would serialize the work here and stall the whole warp on the slowest miss.

Under a skewed access pattern, many threads look up the same hot rows at once. This would mean we'd end up serializing atomic operations for updating bufffer pool replacement metadata on every access. To mitigate this, we plan to deduplicate first, update metadata once per unique row, and try out various replacement policies.

Resources

GHC RTX 2080s

Will confirm GHC host DRAM capacity since it bounds the largest table we can hold

PSC GPU access?

Will write the code from scratch in CUDA and C++ ?

small PyTorch reference implementation of product-key lookup will be correctness check for every kernel

Papers we are using as references:

1. Lample et al. (2019) and Berges et al. (2024) for the product-key memory computation and realistic layer sizes
2. Qureshi et al., BaM (ASPLOS 2023), for the design of a concurrent, GPU-resident software cache that merges redundant requests
3. Chang et al., GMT (ASPLOS 2024), for multi-tier placement between GPU memory and host memory
4. Lee et al., InfiniGen (OSDI 2024), and SparDA (2026), for predicting next-layer accesses to hide transfer latency
5. Megiddo and Modha, ARC (FAST 2003), for skew-aware replacement

full citations in references

Goals and Deliverables

Our main deliverable is a buffer pool that serves a table larger than GPU memory while staying within about 5× (?) of the all-in-GPU upper bound at high skew.

Performance model If a fraction h of row reads hit in device memory, the effective gather bandwidth is

$$B_{\text{eff}} = \left(\frac{h}{B_{\text{device}}} + \frac{1-h}{B_{\text{fetch}}} \right)^{-1}.$$

Will measure.

Plan to achieve

1. A correct concurrent GPU hash table mapping row ids to cache slots, correct batch deduplication by sort and unique, correct per-slot pin counts, and correct replacement policies
2. Three baselines on the same workload
   - the whole table being in GPU memory (upper bound)
   - on-demand fetching of every requested row from host memory
   - the table in CUDA Unified Memory
3. Both miss paths (zero-copy reads and bulk transfers into the cache) plus a simple rule that picks between them based on the miss count for a batch.
4. Evaluation sweeps over
   - some skewed distribution (need to figure out)
   - cache size from 1% to 50 (?)% of the table
   - batch size T and selection size k
   - table sizes from 2^{22} to 2^{24} rows of 1 KB (4 to 16 GB). The 4 GB table still fits on the GPU and gives the all-in-GPU upper bound the 8 and 16 GB tables exceed the 8 GB GPU
5. Performance targets, measured as gathered rows per second
   - Within 5× (?) of the all-in-GPU upper bound on some skewed distribution with a cache of 10% of the table
   - Faster than Unified Memory by a factor we will measure
6. A profile of what limits performance using Nsight

Hope to achieve

- A second platform on PSC Bridges-2
- We would go through router traces from an open MoE model such as OLMoE through our buffer pool to test our design at both 1 KB rows and whole experts
- Speculative prefetching
- Train a small product-key memory model, record its row accesses, and try predicting each layer's lookups from the previous layer's hidden state
- When training, the backward pass scatter-adds gradients into the value table, so offloaded rows become dirty and need to be written back.

If the work goes more slowly

We will have the buffer pool with deduplication and LRU replacement, one miss path (bulk transfers), synthetic traces, and comparison against the all-in-GPU and on-demand fetching baselines.

Questions our analysis will answer

- When does bulk transfer beat zero-copy? Why?
- How much do deduplication and skew-aware replacement raise hit rate and throughput at each skew level?
- Identify the bottlenecks across the evaluation sweep

Platform Choice

The system has to run on a GPU. GHC RTX 2080 makes experimenting easy. The workload is bound by memory from gathering rows, and we want to keep hot rows in device memory. 8 GB on GHC RTX 2080 is a realistic constraint. PSC Bridges-2 H100 or V100 node would be a second platform where device memory is much faster relative to host and would be a good test if our system scales performance across hardware.

Schedule

Week	Goals	Owner
Nov 9–15	Submit proposal. Microbenchmarks for reading device memory, bulk fetching memory from host to device, and zero-copy bandwidth across transfer sizes. Workload generator (skew, reuse across batches). PyTorch reference lookup.	Unmesh
Nov 9–15	The three baselines: all-in-GPU gather, on-demand host fetch, Unified Memory. Confirm GHC host DRAM capacity. Figure out our allocation/use for PSC Bridges-2 GPU nodes?	Sumukh
Nov 16–22	Concurrent GPU hash table, cached gather kernel, batch deduplication, pin counts.	Unmesh
Nov 16–22	Explore replacement policies (like LRU and CLOCK replacement), bulk-transfer miss path, first hit-rate and throughput numbers for the milestone report.	Sumukh
Nov 23–29	Milestone report (Nov 23). Zero-copy miss path and the per-batch choice rule. Frequency-aware or ARC-style policy.	Both
Nov 30–Dec 6	Full evaluation sweeps (skew, cache size, batch size, table size). Nsight profiling of bottlenecks. Stretch: prefetching with a tunable-recall predictor, and one sweep on a Bridges-2 GPU node if possible.	Both
Dec 7–11	Final analysis and report (due Dec 10). Poster for the Dec 11 session, 5:30–8:30 pm.	Both

References

1. G. Lample, A. Sablayrolles, M. Ranzato, L. Denoyer, H. Jégou. Large Memory Layers with Product Keys. *NeurIPS*, 2019.

2. V.-P. Berges, B. Oğuz, D. Haziza, W. Yih, L. Zettlemoyer, G. Ghosh. Memory Layers at Scale. *ICML*, 2025 (arXiv:2412.09764).

3. Z. Huang, Q. Min, H. Huang, D. Zhu, Y. Zeng, R. Guo, X. Zhou. Ultra-Sparse Memory Network. *ICLR*, 2025.

4. X. Cheng et al. (DeepSeek-AI). Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models. arXiv:2601.07372, 2026.

5. Z. Qureshi, V. S. Mailthody, et al. GPU-Initiated On-Demand High-Throughput Storage Access in the BaM System Architecture. *ASPLOS*, 2023.

6. C.-H. Chang, J. Han, A. Sivasubramaniam, V. S. Mailthody, et al. GMT: GPU Orchestrated Memory Tiering for the Big Data Era. *ASPLOS*, 2024.

7. W. Lee, J. Lee, J. Seo, J. Sim. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management. *OSDI*, 2024.

8. SparDA: Sparse Decoupled Attention for Efficient Long-Context LLM Inference. arXiv:2606.04511, 2026.

9. N. Megiddo, D. S. Modha. ARC: A Self-Tuning, Low Overhead Replacement Cache. *FAST*, 2003.